



中山大學
SUN YAT-SEN UNIVERSITY

C++语言IO文件操作

中山大学计算机学院



主讲人：郑贵锋



联系方式：

zhenggf@mail.sysu.edu.cn

目录

CONTENTS

01

I/O的基本概念

02

C与C++的I/O特点

03

C++文件I/O的方法

04

C++20新特性

05

总结

目录

CONTENTS

01

I/O的基本概念

02

C与C++的I/O特点

03

C++文件I/O的方法

04

C++20新特性

05

总结

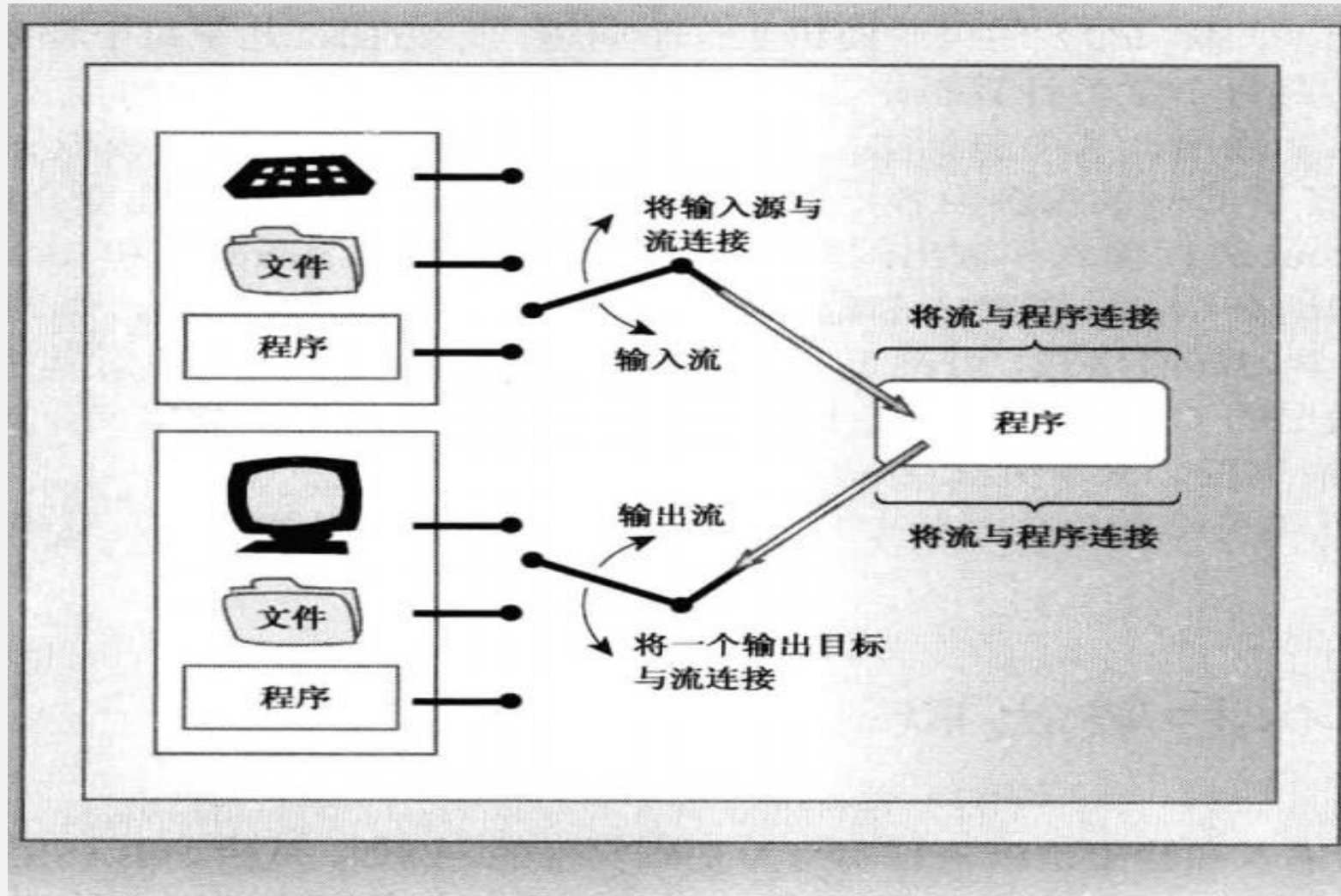


I/O的基本概念

- I/O即Input/Output, 输入输出
- 和C语言一样, C++的I/O同样采取了**流和缓冲区**的概念
- **流**是一种对**连接**的抽象
- 换句话说, 输入流需要**两个连接**, 每端各一个。**文件端连接**提供流的来源, **程序端连接**将流的流出部分转储到程序中 (文件端连接可以是文件, 也可以是设备, 如键盘)。同样, 对输出的管理包括将输出流连接到程序以及将输出目标与流关联起来。这就像将字节 (而不是水) 引入到水管中。



I/O的基本概念





I/O的基本概念

- **流**是一种对**连接**的抽象
- 换句话说，输入流需要**两个连接**，每端各一个。**文件端连接**提供流的来源，**程序端连接**将流的流出部分转储到程序中（文件端连接可以是文件，也可以是设备，如键盘）。同样，对输出的管理包括将输出流连接到程序以及将输出目标与流关联起来。这就像将字节（而不是水）引入到水管中。
- C++程序把输入和输出看作**字节流**。输入时，程序从流中抽取字节；输出时，程序将字节插入到输出流中。
- 输入流中的字节可能来自键盘，也可能来自存储设备或其他程序。
- 同样，输出流中的字节可以流向屏幕、打印机、存储设备或其他程序。
- **流**充当了**程序**和**流源**或**流目标**之间的桥梁。这使得C++程序可以以**相同的方式**对待**来自键盘的输入**和**来自文件的输入**。
- C++程序只检查字节流，而不需要知道字节来自何方。



I/O的基本概念

- **流**已经可以完成输入输出了，缓冲区用来干嘛？
- 流有个最大特点：**流入的量等于流出的量**
- 对于输入源是硬盘来说，一次数据传输的代价是巨大的
- 而程序一般是以一个字节一个字节读取
- 那么硬盘的速度会极大的拉低整个输入的速度
- 因此硬盘一般是一次读取许多数据，缓冲区就作为流的中介，储存这些数据

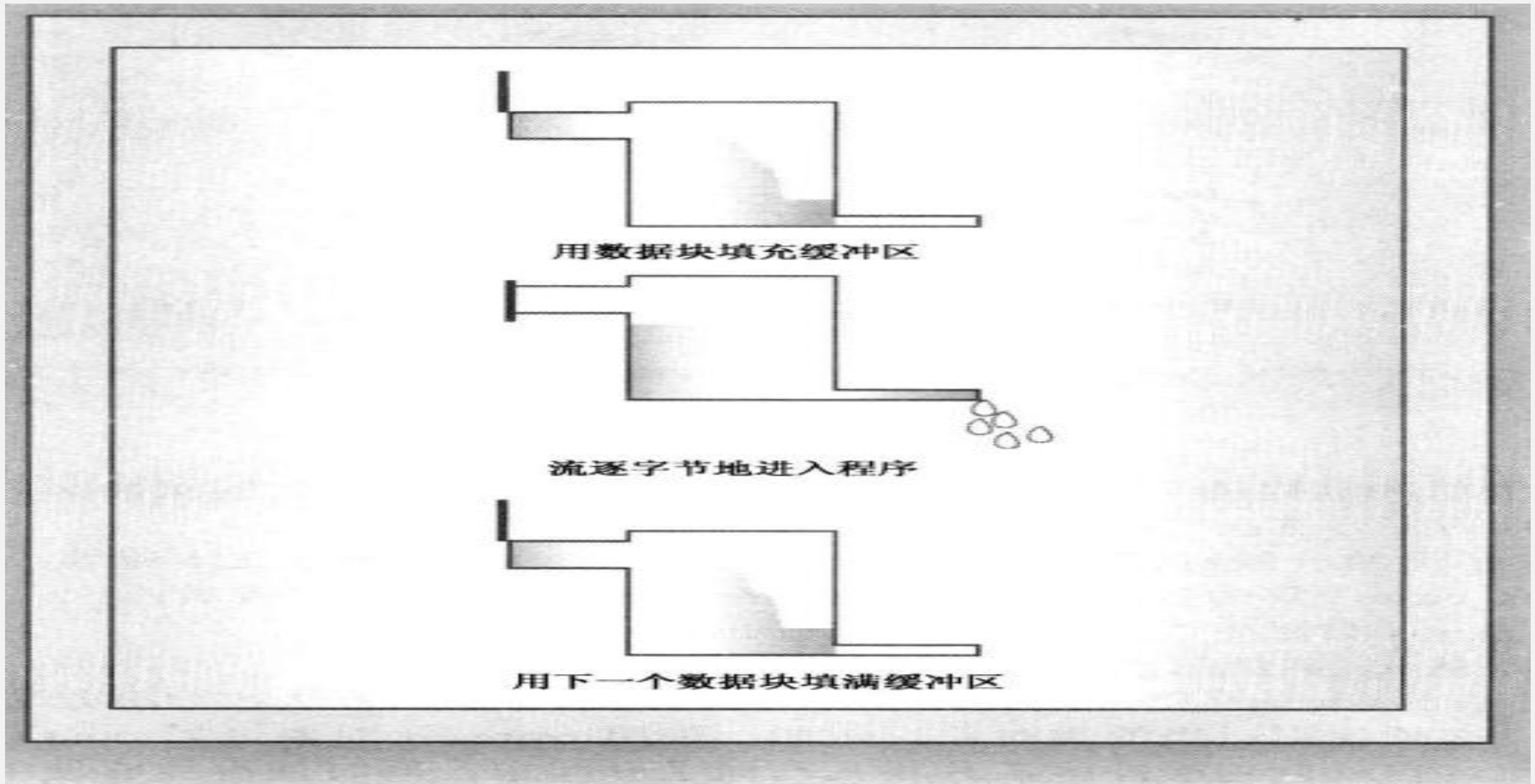


I/O的基本概念

- 缓冲区是用来更高效地处理输入和输出，即存在于输入流和输出流中
- 缓冲区是用作中介的**内存块**，也就是说在内存里
- 它是字节流的一个中介点
- 通常，像磁盘驱动器这样的设备以512字节（或更多）的块为单位来传输数据到内存，而程序通常每次是一个字节一个字节读取。
- 缓存区帮助匹配这两种不同的信息传输速率。
- 例如，假设程序要计算记录在硬盘文件中的金额。程序可以从文件中读取一个字符，处理它，再从文件中读取下一个字符，再处理，依次类推。
- 从磁盘文件中每次读取一个字符需要大量的硬件活动，**速度非常慢**。
- 但有了缓冲区，我们可以从磁盘上读取大量信息，将这些信息存储在缓冲区中，然后每次从缓冲区里读取一个字节。因此从内存中读取单个字节的速度比从硬盘上读取块很多，所以这种方法更快，也更方便。



I/O的基本概念





I/O的基本概念

- 缓冲区的作用？
- 从命令行向程序输入数据时，实际上是输入到缓冲区里
- 一旦按Enter回车键，缓冲区里的数据才流入程序
- 在按回车键之前，我们可以修改缓冲区（即当前行）的数据
- 但一旦按回车键后，我们就不能修改当前行数据了
- 程序输出时，同样是输出到缓冲区里
- 当缓冲区满了，或者用户手动刷新缓冲区（C语言中的`flush(stdout)`），缓冲区里的数据才流出去（比如显示在屏幕上）
- 示例程序
- 有时候，程序非正常结束时，在缓冲区里的数据未能输出而会丢失



I/O的基本概念

- 事实上，在Linux中程序一般存在三个流：**输入流（stdin）**、**输出流（stdout）**、**错误流（stderr）**
- 输入流和输出流有缓冲区
- 错误流是特殊的输出流，专门输出错误信息
- 错误流没有缓冲区，即一有数据立刻输出，避免前面提到的程序非正常结束时的缓冲数据丢失
- 输入流默认来自于标准输入（键盘）
- 输出流和错误流默认流向标准输出（屏幕）

目录

CONTENTS

01

I/O的基本概念

02

C与C++的I/O特点

03

C++文件I/O的方法

04

C++20新特性

05

总结

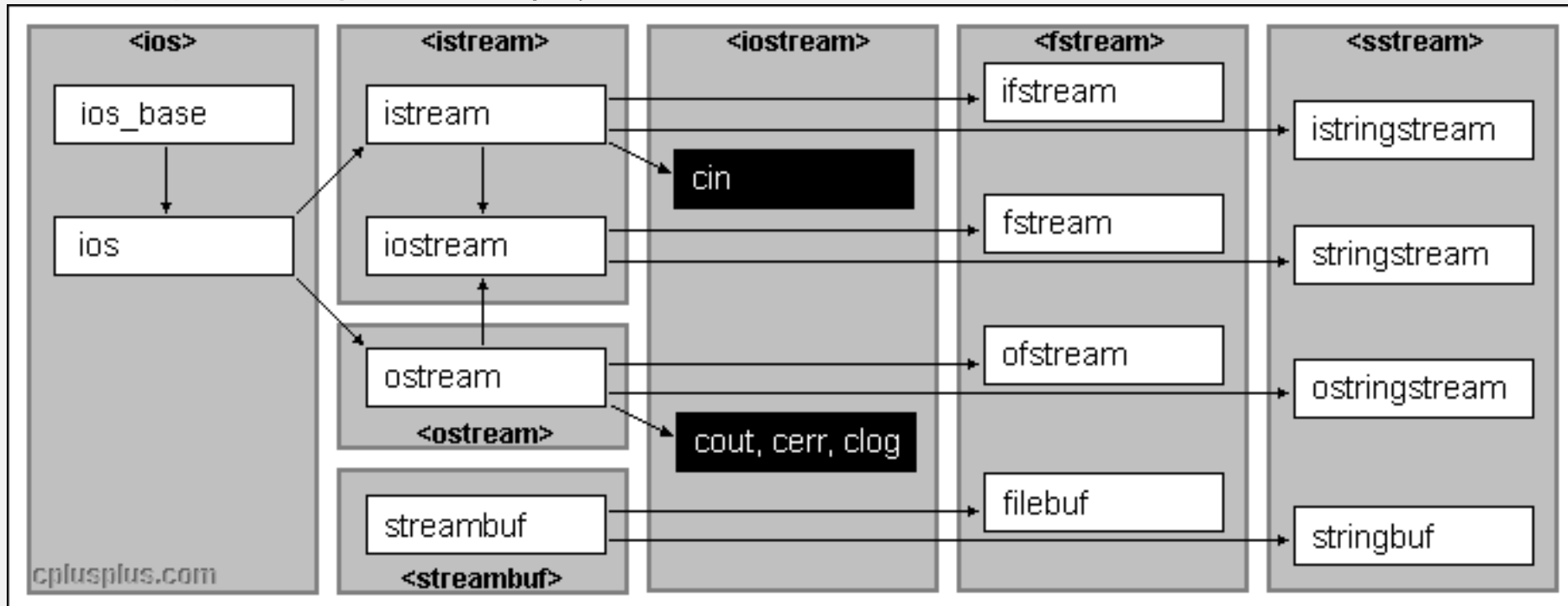


C的I/O特点

- C语言中没有输入输出语句
- 所有的输入输出功能都用 ANSI C提供的一组标准库函数来实现。
- 结构体FILE
- fopen 打开一个文件
- fclose 关闭一个文件
- fgetc 从文件中读取一个字符
- fprintf 往文件中写格式化数据
- fscanf 格式化读取文件中数据
- fread 以二进制形式读取文件中的数据
- fwrite 以二进制形式写数据到文件中去
- feof 文件结束
- ferror 文件读/写出错
- fseek 随机定位

C++的I/O特点

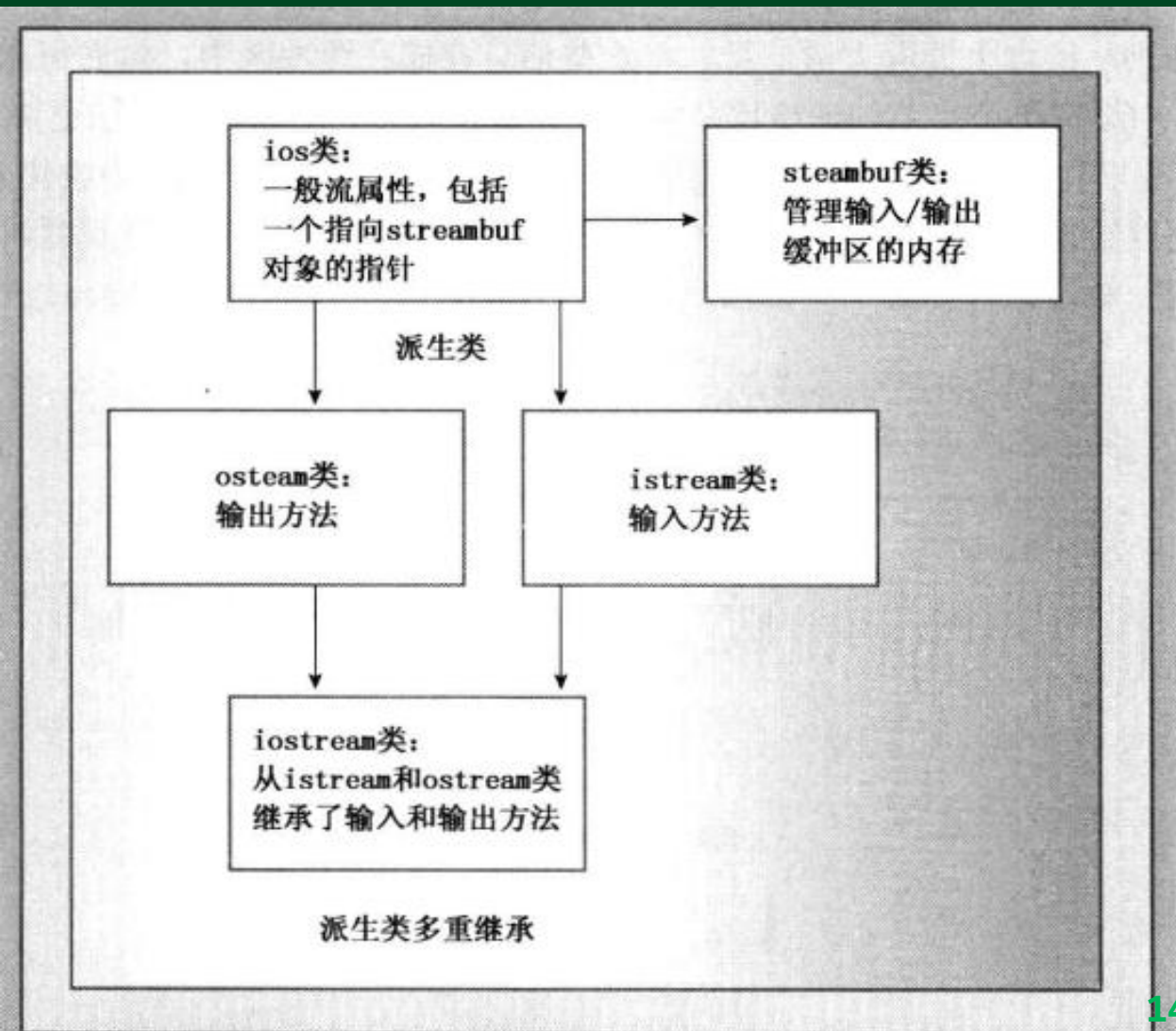
- C++保留了对C中I/O的函数，同时也有一套专属于C++的I/O
- C++使用了很多较为高级的语言特性来实现输入和输出，其中包括：**类、派生类、函数重载、模板和多重继承**。
- 一切文件操作函数都是定义在类里的成员函数！





C++的I/O特点

- streambuf类为缓冲区提供了内存，并提供了用于填充缓冲区、访问缓冲区内容、刷新缓冲区和管理缓冲区内存的类方法；
- ios_base类表示流的一般特征，如是否可读取、是二进制流还是文本流等；
- ios类基于ios_base，其中包括了一个指向streambuf对象的指针成员；
- ostream类是从ios类派生而来的，提供了输出方法；
- istream类是从ios类派生而来的，提供了输入方法；
- iostream类是基于istream和ostream类的，因此继承了输入方法和输出方法。



目录

CONTENTS

01

I/O的基本概念

02

C与C++的I/O特点

03

C++文件I/O的方法

04

C++20新特性

05

总结



C++文件I/O的方法

- **cin**和**cout**实际上是C++中已经定义好的**类的实例**，当然还有**cerr**
- **cin**属于**istream**类，**cout**属于**ostream**类，其中已经重载的两个重要运算符 **>>** 和 **<<** 用来输入和输出。一般从键盘输入和输出到屏幕上。
- C++文件的I/O有以下类：
- **ofstream**: 写操作（输出）的文件类（由**ostream**继承而来）
- **ifstream**: 读操作（输入）的文件类（由**istream**继承而来）
- **fstream**: 可同时读写操作的文件类（由**iostream**继承而来）
- 用法和cin、cout完全一样！



简单例子

```
// FileIO0.cpp
#include <fstream>
using namespace std;

int main () {
    ifstream source("source-file.txt");
    ofstream destination("dest-file.txt");
    int x;
    source >> x;           // Reads one int from source-file.txt
    source.close();         // close file as soon as we 're done using it
    destination << x;       // Writes x to dest-file.txt
    return 0;
}                          // close () called on destination by its destructor
```



解释

- `ifstream source("source-file.txt");`
- 通过帶参数的构造函数，声明了一个类实例，建立了source-file.txt与程序的输入关系
- 也可以调用成员函数open()来建立这个输入关系，即
- `source.open("source-file.txt");`
- 与cin类似，通过>>符号将内容输入给变量x
- `source >> x;`
- 当输入完后，调用类成员函数close()即可关闭文件。相当于C中的fclose()
- `source.close();`
- 如果未调用函数close，在source销毁时调用的析构函数也会自动调用close()



例子1

- 文本文件流—例子1
- 输入文件source.txt，保存在与下面的FileIO.cpp同一个文件夹中，其内容为：
- 13 3.14 9
- 然后编写一程序，把这个文件的内容读入到一个int变量、一个double变量和char变量中。这样这三个变量的值应该分别为13、3.14、'9'。计算前两者之积，然后把结果输出到文件result.txt中保存起来。



示例代码

```
//FileIO1.cpp
//功能: 说明利用文件流实现文件I/O
#include <fstream>           //①
using namespace std;

int main( )
{
    int someInt;
    float someFloat;
    char someChar;
    ifstream inFile;         //②
    ofstream outFile;        //②
    inFile.open("Source.txt"); //③
    outFile.open("Result.txt"); //④
    :
```

```
inFile >> someInt
      >> someFloat
      >> someChar;          //⑤

outFile << "The answer is: "
      <<someInt*someFloat
      <<endl;                //⑥

inFile.close();              //⑦
outFile.close();             //⑦

return 0;
}
```



运行结果

- someInt\someFloat\someChar的值将分别为13\3.14\'9'。
- 在FileIO.cpp同一目录下，出现一个新的文件：result.txt。将其打开，将看到：
- The answer is: 40.82



解释

```
#include <fstream>
```

Step1: 由于类**ifstream**和**ofstream**定义在头文件**fstream**中，所以在文件头需要加上预编译指令。

```
ifstream  inFile;  
ofstream  outFile;
```

Step2: 定义两个对象**inFile**和**outFile**，也称为文件流。前者负责文件输入，后者负责文件输出。



解释

Step3: open语句是把文件流与具体的文件关联起来，使得后续的具体的读写操作作用于这些文件之上。

```
inFile.open(“Source.txt”);
```

把输入文件流**inFile**与文件**source.txt**关联起来，后面从**inFile**输入数据便是从文件**Source.txt**里读取数据；

```
outFile.open(“Result.txt”);
```

把输出文件流**outFile**与文件**result.txt**关联起来，后面输出的结果放入**outFile**中最终就会保存到**result**里面。



解释

```
inFile >> someInt >> someFloat >> someChar;
```

从输入文件流**inFile**中读取数据，置入变量中

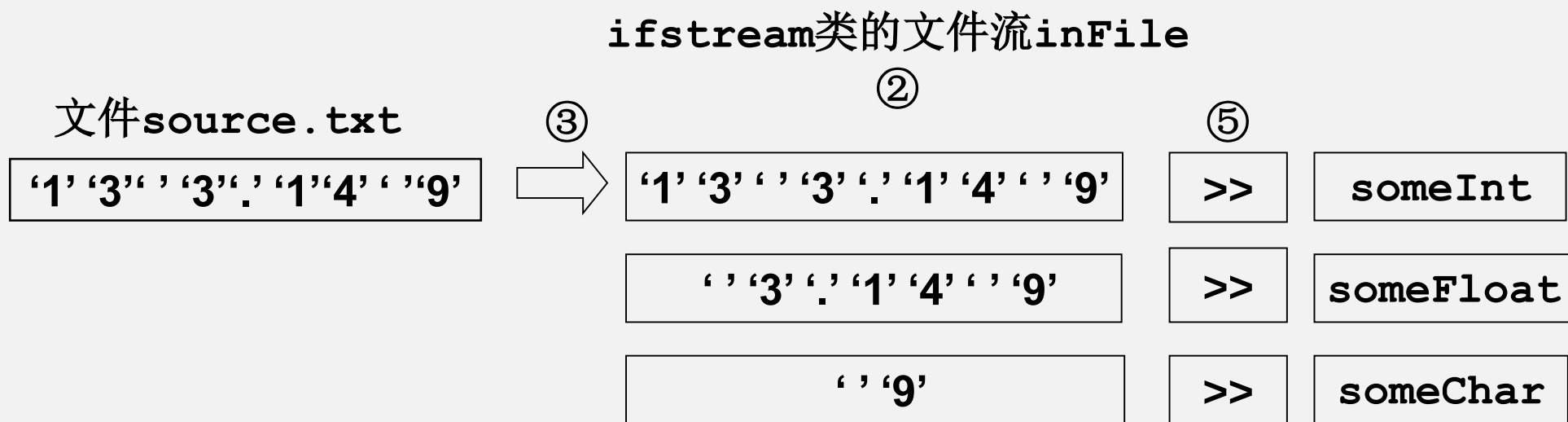
```
outFile << "The answer is: " << someInt * someFloat  
        << endl;
```

把要输出的内容放入输出文件流里面，最终保存到文件
result.txt里面



图解

```
inFile >> someInt >> someFloat >> someChar;
```





图解

```
outFile << "The answer is: " << someInt*someFloat  
<< endl;
```

文件result.txt

'T' 'h' 'e' ' ' 'a' 'n' 's' 'w' 'e' 'r' '
' 'i' 's' ' ' ' : ' ' '

'T' 'h' 'e' ' ' 'a' 'n' 's' 'w' 'e' 'r' '
' 'i' 's' ' ' ' : ' ' ' '4' '0' ' ' ' . ' '8' '2'

ofstream类的文件流outFile②

④
← 'T' 'h' 'e' ' ' 'a' 'n' 's' 'w' 'e' 'r' '
' 'i' 's' ' ' ' : ' ' '

← '4' '0' ' ' ' . ' '8' '2'

⑥
<< "The answer is: "

⑥
<< 40.82



图解

```
outFile << "The answer is: " << someInt*someFloat  
<< endl;
```

文件result.txt

'T' 'h' 'e' ' ' 'a' 'n' 's' 'w' 'e' 'r' '
' 'i' 's' ' ' ' : ' ' '

'T' 'h' 'e' ' ' 'a' 'n' 's' 'w' 'e' 'r' '
' 'i' 's' ' ' ' : ' ' ' '4' '0' ' ' ' . ' '8' '2'

ofstream类的文件流outFile②

④
← 'T' 'h' 'e' ' ' 'a' 'n' 's' 'w' 'e' 'r' '
' 'i' 's' ' ' ' : ' ' '

← '4' '0' ' ' ' . ' '8' '2'

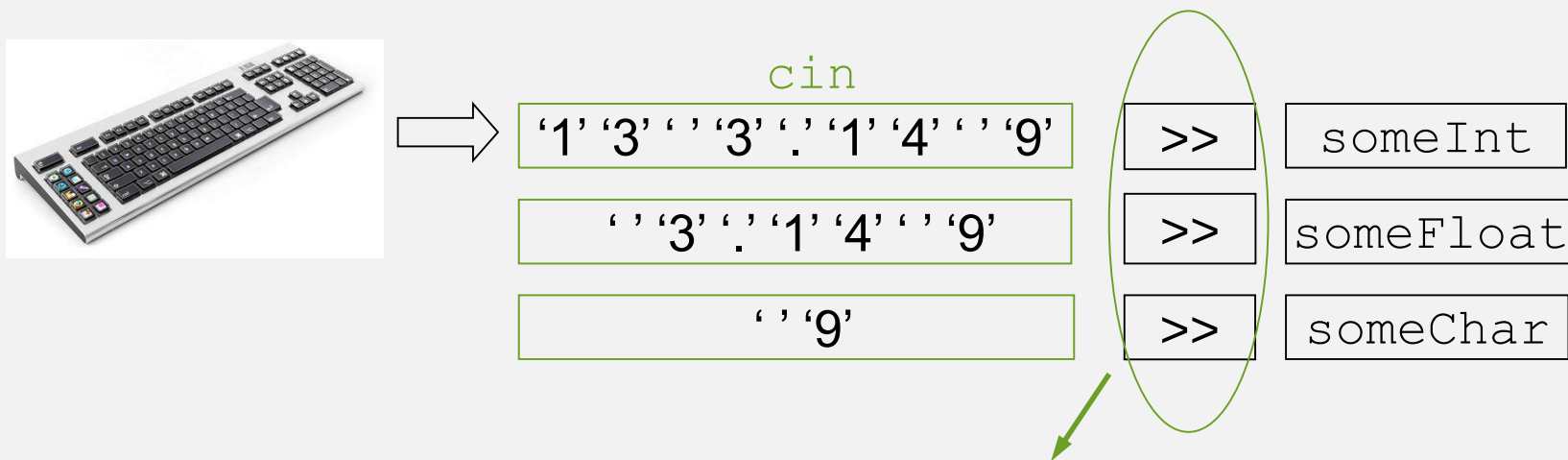
⑥
<< "The answer is: "

⑥
<< 40.82

与键盘输入、显示器输出的比较

```
cin >> someInt >> someFloat >> someChar;
```

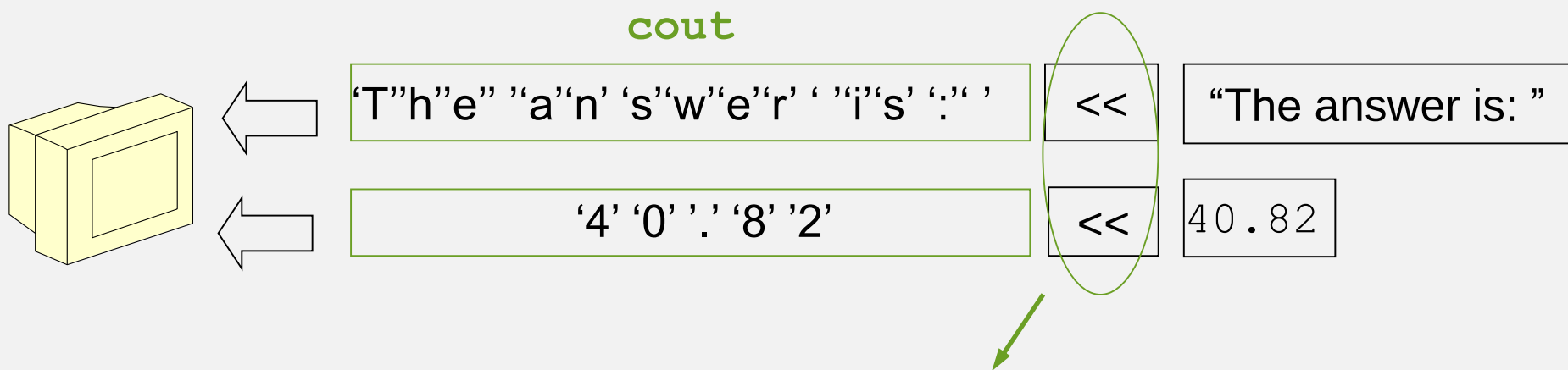
// 假设键盘中敲入 13 3.14 9 ☒



cin缺省情况下是装载字符的数据流，输入符”>>”负责转化的工作。

与键盘输入、显示器输出的比较

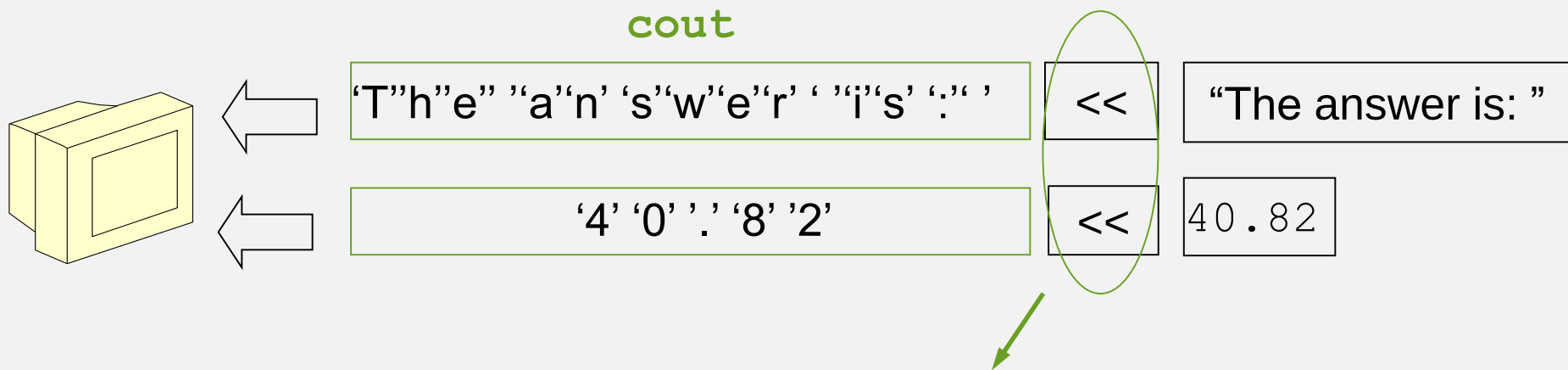
```
cout << "The answer is: " << someInt*someFloat;
```



cout是只能装载字符的数据流，输入符” <<”要负责转化的工作。

与键盘输入、显示器输出的比较

```
cout << "The answer is: " << someInt*someFloat;
```



cout是只能装载字符的数据流，输入符” <<”要负责转化的工作。



例子2

- 假设已有纯文本文件1.txt，现编一程序copy，将1.txt拷贝到另一个纯文本文件2.txt中。



示例代码

```
// FileOI2.cpp
#include <fstream>
using namespace std;

int main( )
{
    char c;
    ifstream  inFile;

    ofstream  outFile;

    inFile.open("1.txt");
    outFile.open("2.txt");
    :
```

```
    inFile >> c;
    while( inFile )
    {
        outFile << c;
        inFile >> c;
    }

    inFile.close();
    outFile.close();

    return 0;
}
```



例子2

- 运行程序，结果如何？出现什么问题？
- 为什么会出现这样的问题？
- 如何解决这个问题？



正确代码

```
//FileIO2Correct.cpp
#include <fstream>
using namespace std;

int main( )
{
    char c;
    ifstream inFile;

    ofstream outFile;

    inFile.open("1.txt");
    outFile.open("2.txt");
    :
```

```
    inFile.get(c);
    while( inFile )
    {
        outFile << c;
        inFile.get(c);
    }

    inFile.close();
    outFile.close();

    return 0;
}
```

文件打开方式

- 在C语言中，我们调用fopen函数，除了文件名外还需要指定打开方式
- 比如“r”表示只读，“w”表示写入，“a”表示追加等

文件使用方式	含义	文件使用方式	含义
“r”（只读）	为输入打开一个文本文件	“r+”（读写）	为读/写打开一个文本文件(要求文件已存在)
“w”	为输出打开一个文本文件	“w+”（读写）	为读/写建立一个新的文本文件
“a”	向文本文件尾追加数据	“a+”（读写）	为读写打开一个文本文件
“rb”	为输入打开一个二进制文件	“rb+”（读写）	为读/写打开一个二进制文件
“wb”	为输出打开一个二进制文件	“wb+”（读写）	为读/写建立一个新的二进制文件
“ab”	向二进制文件尾追加数据	“ab+”（读写）	为读写打开一个二进制文件



文件打开方式

- 同样，在C++的ifstream里的open函数还有第二个参数表示打开方式

```
void open (const char * filename, openmode mode);
```

- 文件的输入输出是从内存的角度看的：
- 数据载入内存叫做输入，数据从内存到其他地方叫做输出
- 对于硬盘的角度来看，是完全相反过来的



文件打开方式

- 同样，在C++的ifstream里的open函数还有第二个参数表示打开方式

```
void open (const char * filename, openmode mode);
```

- mode的表示
- **ios::out** 文件以输出(写)方式打开
- **ios::in** 文件以输入(读)方式打开
- **ios::ate** 初始位置：文件尾
- **ios::app** 所有输出附加在文件末尾
- **ios::trunc** 如果文件已存在则先删除该文件
- **ios::binary** 二进制方式
- 问：
- **ios::** 表示什么意思？



文件打开方式

- 同样，在C++的ifstream里的open函数还有第二个参数表示打开方式

```
void open (const char * filename, openmode mode);
```

- mode的表示
- ios::out** 文件以输出(写)方式打开
- ios::in** 文件以输入(读)方式打开
- ios::ate** 初始位置：文件尾
- ios::app** 所有输出附加在文件末尾
- ios::trunc** 如果文件已存在则先删除该文件
- ios::binary** 二进制方式

```
ofstream file;
```

```
file.open ("example.bin", ios::out | ios::app | ios::binary);
```

- 以输出、追加、二进制方式打开，等价于以下方式

```
ofstream file("example.bin", ios::out | ios::app | ios::binary);
```



文件打开方式

- 同样，在C++的ifstream里的open函数还有第二个参数表示打开方式

```
void open (const char * filename, openmode mode);
```

- mode的表示
- **ios::out** 文件以输出(写)方式打开
- **ios::in** 文件以输入(读)方式打开
- **ios::ate** 初始位置：文件尾
- **ios::app** 所有输出附加在文件末尾
- **ios::trunc** 如果文件已存在则先删除该文件
- **ios::binary** 二进制方式

```
ofstream file;
```

```
file.open ("example.bin", ios::out | ios::app | ios::binary);
```

- 问：
- “|” 是什么？



文件打开方式

- `ios::out` 文件以输出(写)方式打开
- `ios::in` 文件以输入(读)方式打开
- `ios::ate` 初始位置：文件尾
- `ios::app` 所有输出附加在文件末尾
- `ios::trunc` 如果文件已存在则先删除该文件
- `ios::binary` 二进制方式
- 实际上就是一个数字，在源码中，

```
static constexpr _Openmode in          = static_cast<_Openmode>(0x01);
static constexpr _Openmode out         = static_cast<_Openmode>(0x02);
static constexpr _Openmode ate         = static_cast<_Openmode>(0x04);
static constexpr _Openmode app         = static_cast<_Openmode>(0x08);
static constexpr _Openmode trunc       = static_cast<_Openmode>(0x10);
static constexpr _Openmode _Nocreate   = static_cast<_Openmode>(0x40);
static constexpr _Openmode _Noreplace = static_cast<_Openmode>(0x80);
static constexpr _Openmode binary      = static_cast<_Openmode>(0x20);
```



文件打开方式

- **ofstream**, **ifstream** 和 **fstream** 所有这些类的成员函数 **open** 都包含了一个默认打开文件的方式，这三个类的默认方式各不相同：
- 类参数的默认方式
- **ofstream** `ios::out | ios::trunc`
- **ifstream** `ios::in`
- **fstream** `ios::in | ios::out`



例子

- `ofstream file ("example.bin", ios::out | ios::app | ios::binary);`
- 例如：以二进制输入方式打开文件c:\config.sys
- `fstream file1;`
- `file1.open("c:\\config.sys",ios::binary|ios::in,0);`
- //如果open函数只有文件名一个参数，则是以读/写普通文件打开，即：
- `file1.open("c:\\config.sys");<=>file1.open("c:\\config.sys",ios::in|ios::out,0);`
- //另外，fstream还有和open()一样的构造函数，对于上例，在定义的时候就可以打开文件了：
- `fstream file1("c:\\config.sys");`
- 通过调用成员函数`is_open()`来检查一个文件是否已经被顺利的打开，它返回一个布尔(bool)值，为真（true）代表文件已经被顺利打开，假(false)则相反。



例子

- **fstream**类的**open**成员函数有第三个参数**access**，用于标识打开文件的属性取值。
- 0：普通文件，打开访问
- 1：只读文件
- 2：隐含文件
- 4：系统文件
- 可以通过+或者|运算进行组合。
- 感兴趣的可以上网搜寻相关资料

```
void open(const char* filename,int mode,int access);
```



文件I/O

- 所以，在实际应用中，根据需要的不同，选择不同的类来定义：
- 如果想以输入方式打开，就用**ifstream**来定义；
- 如果想以输出方式打开，就用**ofstream**来定义；
- 如果想以输入/输出方式来打开，就用**fstream**来定义。
- 很多程序中，可能会碰到**ofstream out("Hello.txt")**, **ifstream in("...")**, **fstream foi("...")**这样的使用，并没有显式的去调用**open()**函数就进行文件的操作，直接调用了其默认的打开方式，



ios::binary

- C语言中，“wb”或者“rb”是以二进制形式打开文件，并通过**fread()**和**fwrite()**函数进行二进制形式的读入和输出
- 二进制形式即将数据在内存中的样子储存在文件中，一般形式会转换成相应的ASCII码储存
- 与之相对的，C++中用类成员函数**read()**和**write()**进行读写



ios::binary

```
#include <fstream>                                //FileIOBinary.cpp
#include <iostream>
using namespace std;

int main( )
{
    int a[10]={10,20,30,40,50,60,70,80,90,100};
    int b[10], i;
    ifstream inFile;                                //①
    ofstream outFile;                                //①

    outFile.open( "1.dat", ios::binary );            //②
    for( i = 0; i < 10; i++ )
    {
        outFile.write( (char*)&a[i], sizeof( a[i] ) ); //③
    }
}
```



ios::binary

```
inFile.open( "1.dat", ios::binary );           //②
for( i = 0; i < 10; i++ )
{
    inFile.read( (char*)&b[i], sizeof( &b[i] ) ); //⑤
}
inFile.close();                                //④

for ( i = 0; i < 10; i++)
    cout << b[i] << " ";

cout << endl << endl;

return 0;
}
```



二进制输出且追加

```
int main( )                                //FileIOBinaryAppend.cpp
{
    int a[10]={10,20,30,40,50,60,70,80,90,100};
    int b[10];
    int i;
    ifstream inFile;
    ofstream outFile;

    outFile.open( "c:\\1.dat", ios::binary | ios::app );
    for( i = 0; i < 10; i++ )
    {
        outFile.write( (char*)&a[i], sizeof( a[i] ) );
    }
    outFile.close();
    return 0;
}
```



文件定位

- C语言中，我们可以通过fseek()函数改变当前的文件指针位置，即当前指向文件内容的位置
- 与之相对的，C++中用类成员函数seekg()和seekp()进行调整。
- 和C的文件操作方式不同的是，C++ I/O系统管理两个与一个文件相联系的指针。一个是读指针，它说明输入操作在文件中的位置；另一个是写指针，它下次写操作的位置。每次执行输入或输出时，相应的指针自动变化。所以，C++的文件定位分为读位置和写位置的定位，对应的成员函数是 seekg()和 seekp(), seekg()是设置读位置， seekp()是设置写位置。



文件定位

```
istream &seekg(streamoff offset, seek_dir origin);
```

```
ostream &seekp(streamoff offset, seek_dir origin);
```

- streamoff 定义于 `iostream.h` 中，定义有偏移量 `offset` 所能取得的最大值，`seek_dir` 表示移动的基准位置，是一个有以下值的枚举：
- `ios::beg` 文件开头
- `ios::cur` 文件当前位置
- `ios::end` 文件结尾

```
file1.seekg(1234, ios::cur);    //把文件的读指针从当前位置向后移1234个字节
```

```
file2.seekp(1234, ios::beg);    //把文件的写指针从文件开头向后移1234个字节
```

- 我们可以通过类成员函数 `tellg()` 和 `tellp()` 来获取当前读指针和写指针的位置



示例代码

```
#include <iostream>                                     // FileIOSeek.cpp
#include <fstream>
using namespace std;
int main()
{
    int a[10]={10,20,30,40,50,60,70,80,90,100};
    int b[10];
    int i;
    ifstream inFile;
    ofstream outFile;

    outFile.open( "1.dat", ios::binary );                 //②
    for( i = 0; i < 10; i++ )
    {
        outFile.write( (char*)&a[i], sizeof( a[i] ) );
    }
    outFile.close();                                     //④
```



示例代码

```
inFile.open( "1.dat", ios::binary );           //①

for( i = 0; i < 10; i++ )
{
    inFile.read( (char*)&b[i], sizeof( &b[i] ) ); //②
    inFile.seekg( sizeof(int), ios::cur );         //③
}

inFile.close();                                //④

for ( i = 0; i < 10; i++ )
    cout << b[i] << " ";

return 0;
}
```



动态输入文件名

```
#include <fstream>                                     //FileIODIYName.cpp
#include <string>
using namespace std;

int main( )
{
    int a[10]={10,20,30,40,50,60,70,80,90,100};
    int i;
    ifstream inFile;
    string FileName;

    cout << "Please enter the output file name";
    cin  >> FileName;
    outFile.open( FileName.c_str( ), ios::binary );
    :
}
```


目录

CONTENTS

01

I/O的基本概念

02

C与C++的I/O特点

03

C++文件I/O的方法

04

C++20新特性

05

总结

C++20新特性: Format

- 在C中我们可以通过**printf**函数进行格式化输出

```
printf("[ %1f, %1f, %1f ]\n", a[0], a[1], a[2]);
```

- 而如果在C++中我们需要……

```
std::cout << "[ " << a[0] << ", " << a[1] << ", " << a[2] << " ]";
```

- 如果涉及到精度控制的话, 还需要**fixed** **setprecision**等流算子进行控制
- 而如果用C++20中新引进的标准库Format只需

```
std::cout << std::format("[ {}, {}, {} ]\n", a[0], a[1], a[2]);
```

- 非常类似与Python的格式化输出, 它比cout更看得懂, 比printf更加易写
- 事实上**format**函数返回的是一个**std::string**类型

目录

CONTENTS

01

I/O的基本概念

02

C与C++的I/O特点

03

C++文件I/O的方法

04

C++20新特性

05

总结



总结

- 流是进出程序的**字节流**。
- 缓冲区是**内存中的临时存储区域**，是程序与文件或其他I/O设备之间的桥梁。
- 信息在缓冲区和文件之间传输时，将使用设备处理效率最高的尺寸以**大块数据**的方式进行传输。
- 信息在缓冲区和程序之间传输时，是**逐字节传输**的，这种方式对于程序中的处理操作更为方便。
- C++通过**ifstream, ofstream, fstream**类及其一系列的成员函数对文件进行操作
- **open、close、>>、<<、seekp、seekg、read、write**
- 学语言，重点是学习它的抽象，它对行为的抽象，对操作系统提供的API的抽象
- 事实上C++的文件读写，是对操作系统提供的文件读写功能的抽象，即最后都是交由操作系统进行文件内容的读取和写入
- 而操作系统又是对各个硬件的读写进行了抽象，最后还是交由各个硬件的驱动程序进行文件内容的读取和写入